Improving the Usability of Refactoring Tools for Software Change Tasks

Anna Maria Eilertsen

Thesis for the degree of Philosophiae Doctor (PhD)
University of Bergen, Norway
2021

UNIVERSITY OF BERGEN

Chapter 2

Background

Mind the Gap

— *Transport for London*

Refactoring has an almost thirty year history of study. In this chapter, I provide an quick introduction to refactorings and refactoring tools before presenting an overview of early work in refactoring in broad strokes, emphasizing the ideas that shaped the later research. Then, I provide a more detailed description of work related to the usability of refactoring tools for software changes and studies of how developers refactor. I present an overview of the factors that have emerged from previous works as potentially impacting developers' choice of using or not using refactoring tools. And finally, I discuss studies of software change tasks as they relate to the topics in this dissertation.

An early version of the discussion in this chapter was published elsewhere [36].

2.1 Refactoring: A Primer

This thesis relies upon knowledge of refactoring operations and refactoring tools. Here, I provide a short primer on these topics. Additional relevant background can be found in other literature [94, 47].

To *refactor* a program is to change its internal structure without altering its observable behavior [94, 47]. Typically, refactorings are performed to make the program easier to understand or change [47, 64, 112]. For example, the meaning of a function may be clarified by updating its name. Developers often refactor programs in the context

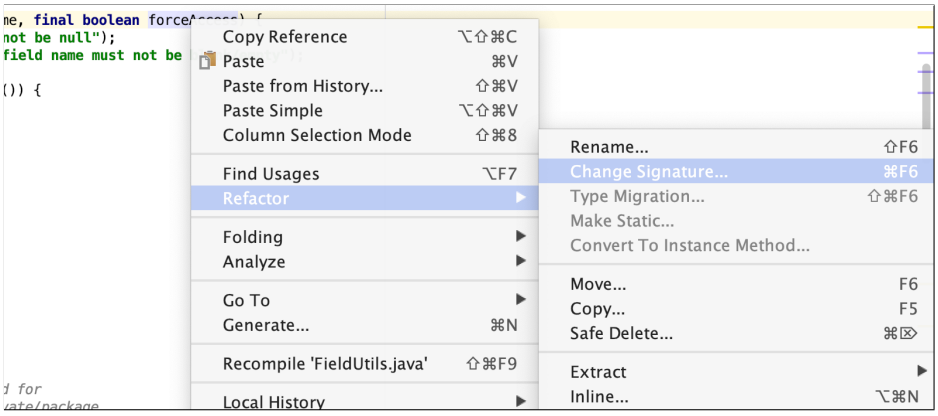

Figure 2.1: IDEs like IntelliJ give developers access to invoke a refactoring menu directly from the editor by right-clicking on a source code element. Rather than showing all the refactorings that the IDE implements, the menu is populated with the refactorings that are possible to invoke on the element that was clicked. Here, the menu was invoked by clicking on a parameter of a method.

of a functional change [86, 74, 95, 35]. Despite the refactoring itself being behavior-preserving, the overall goal of the change might be to implement new functionality or fix a bug [104]. For example, if the behavior of a function is changed, its name might need to be altered accordingly.

A *refactoring operation* is a specific change pattern that often occurs during changes to object-oriented[1] programs [94, 47]. For example, a developer can apply the `rename` operation to a method declaration to clarify its meaning. However, it is not enough to change the declaration: in order to retain the program's meaning, all callers must also be updated to use the new name. If the developer makes this change manually, they must locate and update all callers [47]. However, if the developer initiates the `rename` operation with a refactoring tool, then the tool can make the updates automatically.

Many refactoring operations are automated by *refactoring tools* found in modern mainstream IDEs such as IntelliJ [3] and Eclipse [1]. Figure 2.1 shows one refactoring menu from IntelliJ[2]. Refactoring tools support software change tasks by automatically performing steps of the refactoring process that developers would otherwise have to do manually. For example, the tool can automatically update all locations that are impacted by a refactoring, such as callers in the previous `rename` example. This is especially useful in cases where the initial change impacts many locations (e.g., in the case of applying `rename` to a method with many callers located in different files). In this case, a tool can save the

[1]When the program is not object-oriented, these changes are typically called *restructuring* rather than refactoring [55].

[2]Throughout this thesis, I will use examples from the JetBrains IntelliJ IDE because it is representative of the support in mainstream IDEs. JetBrains is known for producing high-quality IDE support for refactoring such as in the IntelliJ IDE for Java and the Visual Studio plugin ReSharper for C#.

developer from slow and error-prone navigation, editing, and context-switching.

While this depiction of refactorings might evoke associations to simple textual find-and-replace tools, implementing such tools are significantly more difficult. Consider a developer that wants to `rename` a variable named `i` to be called `foo`. A tool that replaces all `i`s in the entire program text, with the string foo would not be considered useful as, for example, every reference to the `int`-type would now be replace with the nonsensical `foont` type. To ensure that a refactoring tool changes the correct code, it needs to operate on a parsed version of the program. This typically entails operating an abstract syntax tree (AST) or a similar data structure, like the Program Source Interface (PSI) type used by IntelliJ [59]. The refactoring tool typically changes this AST or PSI representation of the program and the textual source code is updated accordingly.

To ensure that the changes are safe and correct, a refactoring tool typically performs *condition checks* [94, 22, 42]. Consider again the case of the `rename` operation on a method. If a developer attempts to give the method a name that already is defined in that scope, for example the name of a pre-existing field, then the resulting program will fail to compile. Whilst this might be difficult to identify in a manual process until the refactoring is already applied, due to mechanisms like inheritance, a tool can easily check this before the code is changed. If the name is already there, the tool can stop the application and alert the programmer that the new method signature will collide with an existing element.

For a running example of a refactoring, I introduce the `remove-parameter` refactoring[3]. This refactoring removes a parameter from a method declaration and updates all callers correspondingly. Its conditions are that the parameter is not used in the method's body and that the new signature does not collide with any other method signatures. Listing 2.1 shows a simple example of such a program and Listing 2.2 shows the result of applying `remove-parameter` to that program.

[3] `remove-parameter` is sometimes considered part of `change-signature` [47, 3] and invoked through the `change-signature` menu option.

Listing 2.1: A method `bar` with two parameters, and one caller, `foo`. The parameter `Y y` is unused.

```
60 public boolean foo(A a) {
61    return bar(a.getX(), a.getY());
62 }
63 public boolean bar(X x, Y y) {
64    return x.isBar();
65 }
```

Listing 2.2: The unused parameter `y` was removed from `bar` and from its call in `foo`.

```
60 public boolean foo(A a) {
61    return bar(a.getX());
62 }
63 public boolean bar(X x) {
64    return x.isBar();
65 }
```

If a developer wants to invoke a refactoring tool from an IDE, they can access a refactoring menu like the one shown in Figure 2.1 and select the desired refactoring from the list. Next, a Configuration View is presented, as shown in Figure 2.2. This view lets the developer configure the `change-signature` refactoring as a `remove-parameter` refactoring. This view is typically shown as a popup over the code editor and blocks the developer from making any changes to the source code while they are interacting with the view.

Once the developer finishes the configuration, she can Preview the refactoring to see a list of locations that will be changed, as shown in Figure 2.3. The developer may also click Refactor to immediately apply the entire change without previewing it. In both cases, the tool will check that refactoring's conditions and present a problem view if it detects any violations. Figure 2.4 shows such a view for the `remove-parameter` refactoring. Also in this case, the developer can apply the refactoring by clicking Continue in Figure 2.4.

In the Preview View, it is often possible to navigate to the identified locations and to exclude locations from being updated, as shown in Figure 2.3. When the developer is satisfied with the Preview, she may click Do Refactor to apply the refactoring. While some implementations of Preview View (like Eclipse) prevent the developer from interacting with the code, the one shown here, from IntelliJ, affords the developer the opportunity

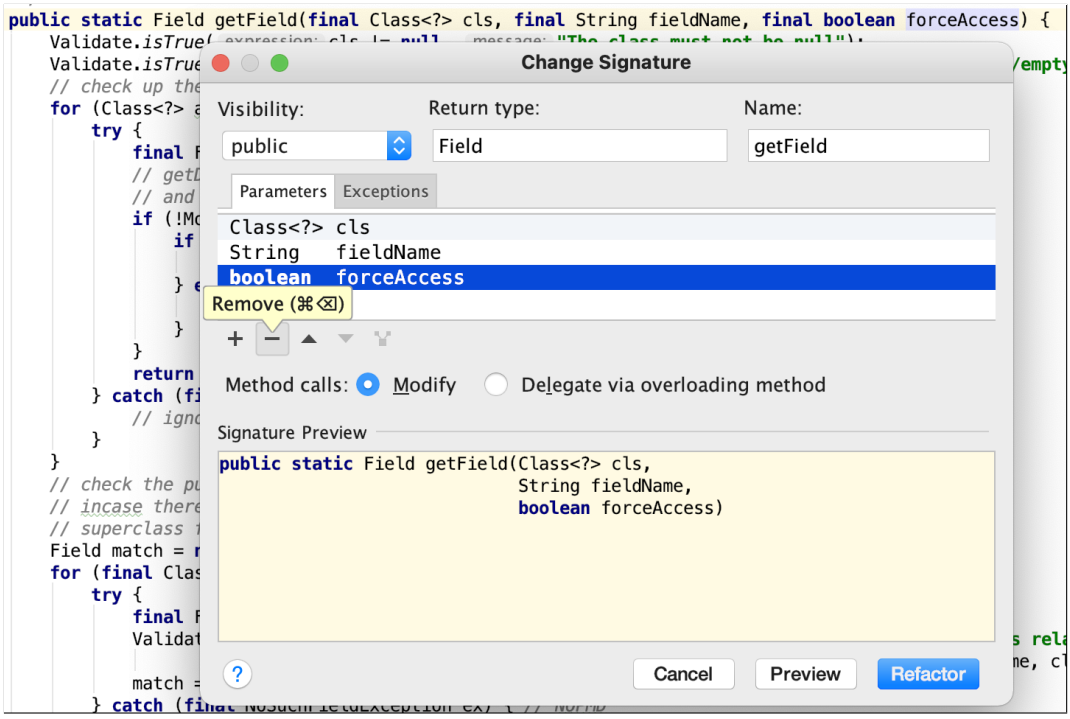

Figure 2.2: The configuration view of `change-signature` in IntelliJ. Here, the developer can configure the refactoring tool to apply `remove-parameter`. Once the developer has finished the tool's configuration, she can click Refactor to make the tool attempt to apply the refactoring. She can also choose to Preview the configured refactoring operation or use Cancel to discard the refactoring. While this view is shown, it is not possible to edit the source code shown in the editor behind the popup.

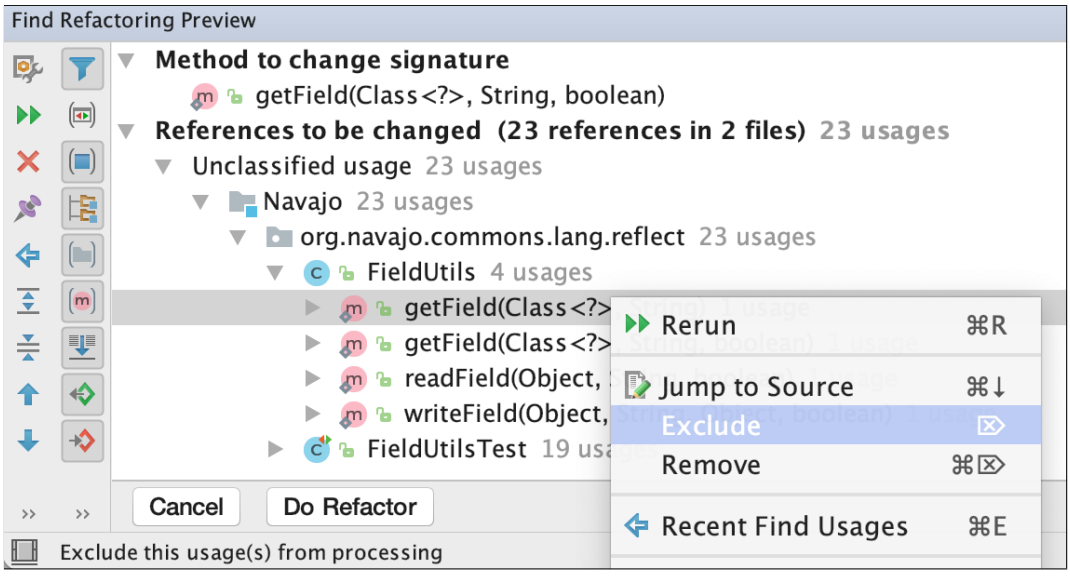

Figure 2.3: IntelliJ's Preview view for an application of `change-signature` to the method `getField`. The tool is configured to remove a parameter from the method. This Preview View shows all the 23 references that needs to be updated for the refactoring to preserve behavior. For each reference, a user may choose to "exclude this usage from processing". The Preview View is shown in a paneø that allows editing the source code; however, doing so will render the refactoring invalid and raise an error.

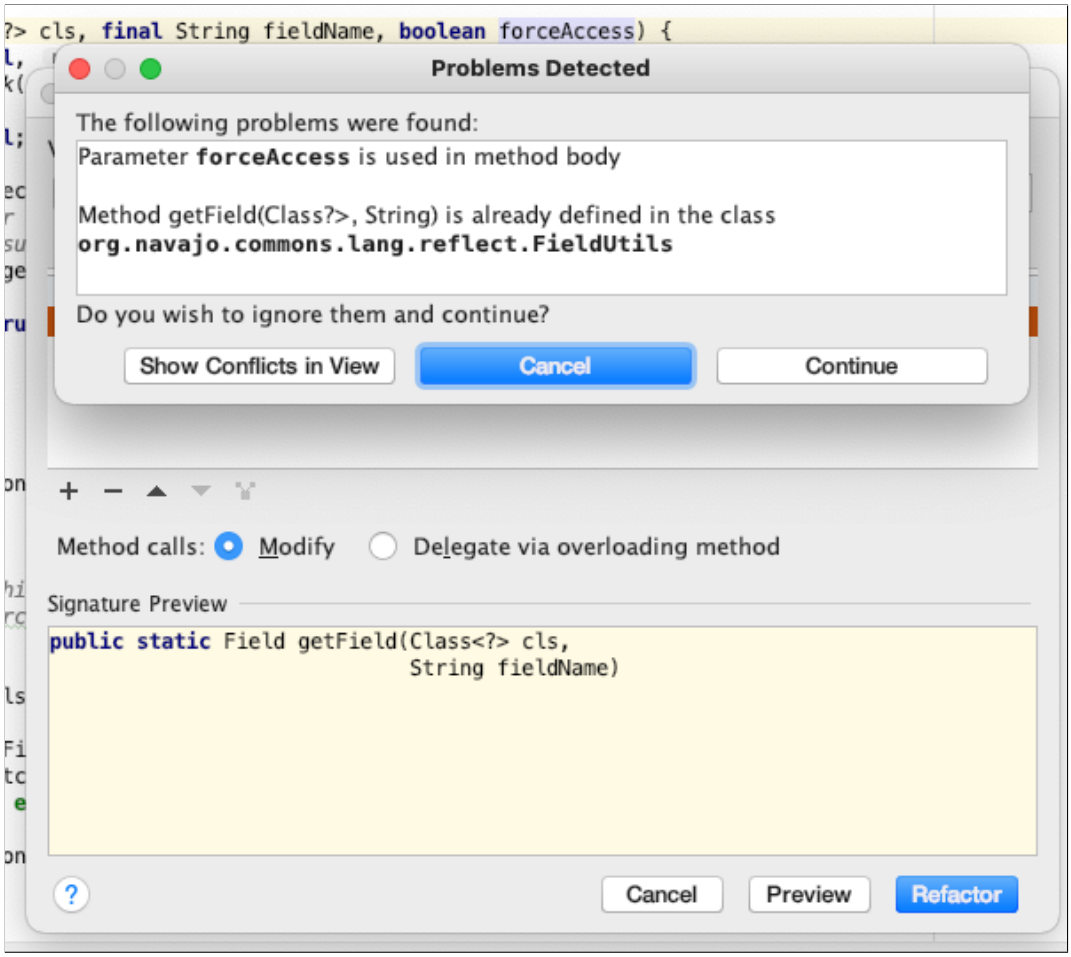

Figure 2.4: Problem view for the `change-signature` as a popup over the configuration view. The condition checker has found two problems, both of which are going to lead to compiler errors if the refactoring is applied. The user is given the option to learn more, to cancel, or to continue the application despite the problems.

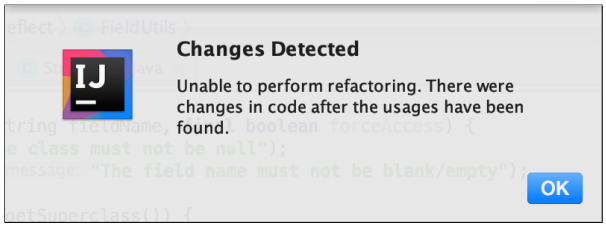

Figure 2.5: If a developer alters the source code during the preview step of a refactoring in IntelliJ, that refactoring can no longer be applied. Instead, this error is produced, and the refactoring must be re-invoked and re-configured before application.

to alter the source code before continuing the refactoring. However, if they choose to do so, the refactoring tool will prevent the application of the refactoring. In this case, the developer is forced to re-initiate and re-configure the refactoring. Figure 2.5 shows the error message that IntelliJ presents in such a situation. This is due to the tool's reliance of the code being unchanged to ensure that the refactoring and the conditions that has been checked are still valid.

Once the refactoring is applied, the tool changes all the identified locations in the source code according to its implementation, except any excluded locations.

Figure 2.6 illustrates the refactoring tool workflow supported by refactoring tools in IDEs. It shows that after a developer invokes and configures a refactoring tool, the tool transforms the source code according to that tool's implementation. As the exact transformation can vary [93], developers may be unable to correctly predict the outcome of a tool invocation [92] and risk applying transformations that they did not intend [86, 126]. It can even be difficult to inspect or verify the changes after they happen [40]. Regardless of how complex and distributed a change is, the tool performs the transformation in a single, atomic step. The resulting all-or-nothing situation forces the developer to choose between applying one atomic change for which they cannot predict the outcome, or editing the code manually.

As indicated in Figure 2.3 and Figure 2.4, the mainstream refactoring tool lets the user apply unsafe refactorings. In Figure 2.3, the user can exclude references from being updated. In this case, these references might change the program's behavior, either by preventing compilation or by altering references. For example, if a parameter is removed and one caller is excluded, as indicated in Figure 2.3, that caller will no longer bind to the right method and the program will not compile. If a removed parameter is referred to in the method body, as shown in Figure 2.4, a similar problem will arise. As another possibility, if there existed a field with the same name in that class, the program might still compile, but its behavior might have been changed.

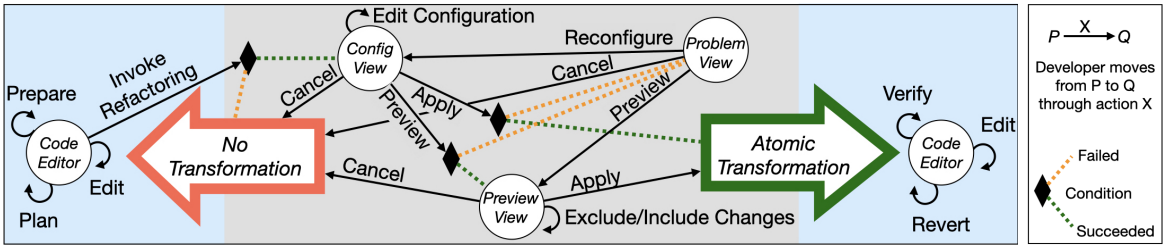

Figure 2.6: The design of most refactoring tools makes the developer "leave" the code editing context (shown in blue) when invoking a tool and "return" either by applying an entire transformation (right green arrow) to the source code or by discarding the change (left red arrow). While interacting with the tool (gray box), the developer can inspect and edit the upcoming refactoring (i.e., navigate the black lines). The Config(uration) View is used to edit the refactoring. The tool can check for problems (⧫) and inform the user about violated conditions (Problem View) and about the locations that will be impacted (Preview View). If the change is discarded, no configurations, transformations, or other information is retained.

2.2 Refactoring Tools: the Beginnings

The first organized examination of refactorings were done by Opdyke [94] and Griswold [55] in their dissertations in the early 1990s. Griswold described a number of restructurings that could ease program maintenance without changing a program's behavior, such as `rename`, `extract-method`, and `inline-method`. He also demonstrated that these types of transformations could be automated. Opdyke studied the evolution of object-oriented programs in C++ and created a catalogue of restructurings, or *refactorings*, that frequently occurred and were useful during the program's evolution. Included in the catalogue were the conditions under which each refactoring could be applied to a program written in C++ without altering its behavior. Shortly after, Roberts, Brant, and Johnson implemented a number of these refactorings and their conditions in the Smalltalk Refactoring Browser [104, 105].

These early works shaped much of the later research in two ways. Firstly, the specific refactorings that were described, catalogued, and automated, are still largely the same that are found in modern IDEs today [1, 3]. Secondly, the emphasis on preserving program behavior is still considered as essential to the refactoring discipline. Despite the incompleteness of Opdyke's conditions [120], the idea of combining code transformation with correctness checks has prevailed. In fact, the refactoring tools that are found in modern IDEs also use condition checks to guard against erroneous applications.

The interest in refactoring was fueled by its promise of a faster and cheaper means of evolving and improving object-oriented software. This was illustrated in Tokuda and Batory's study of refactoring-aided software evolution [120]. They selected two programs

that had been manually evolved by developers, and for each program, they selected two versions of their design, the source and target state. Then, they attempted to use refactoring operations to transform the source state into the target state. Based on this study, they estimated how much time could have been saved if the two programs had been evolved with the aid of tools that could automatically apply the refactorings rather than with a manual approach. For one program, they estimated that the time could be reduced from two days to two hours and for the other program, from two weeks to one day. The possibility of such drastic improvements on developer's productivity was an enticing prospect and motivated the development of automated refactoring tools.

There are substantial technical challenges associated with implementing refactoring tools [104]: they require non-trivial program analysis and transformation capabilities[59]. The details of correctness conditions and program transformations vary between languages which means that is difficult to reuse tool components written for one programming language in a tool targeting another. One example of this is the Refactoring Browser. This was an early example of automated refactoring support but only supported Smalltalk. This tool could not be used to refactor programs that were written in other languages, such as Java and C++.

In the late 1990s, there was consequently a sizable demographic of Java programmers in the industry who had little or no support for automated applications of refactorings. Instead, these practitioners were introduced to refactoring operations through Fowler's seminal book, Refactoring: Improving the Design of Existing Code [46]. This book described how and when to apply a catalogue of refactoring operations. While this catalogue largely contained the same ones as those described by Opdyke and Griswold and the ones implemented in the Smalltalk tool, the descriptions were aimed at practitioners without tool support. The book used a natural language aimed at practitioners to describe how to manually apply these operations. It also emphasized that the program's behavior should be preserved, but it recommends achieving this through small and controlled code changes and frequently running a test suite.

In 2004, Mens and Tourwé presented the first survey of software refactorings [78] and introduced a model of the refactoring process:

1. Identify where the software should be refactored.

2. Determine which refactoring(s) should be applied to the identified places.

3. Guarantee that the applied refactoring preserves behavior.

4. Apply the refactoring.

5. Assess the effect of the refactoring on quality characteristics of the software (e.g., complexity, understandability, maintainability) or the process (e.g., productivity, cost, effort).

6. Maintain the consistency between the refactored program code and other software artifacts (such as documentation, design documents, requirements specifications, tests, etc.).

At this point, there was already substantial research into how to implement and improve refactoring tools. This research focused largely on technical improvements of the tool's ability to perform one or more of these steps. For example, researchers attempted to create refactoring recommendation tools that could support steps 1-2 [62], worked on implementing a wider variety of refactorings [107, 73, 104] (step 4) and implemented condition checks [30, 119, 79] (step 3). Condition checks are important because they help ensure that the program's behavior is preserved. However, Mens and Tourwé described that, in many cases, a formal guarantee for behavior preservation would cause tools to be overly restrictive; Brant later made a similar observation [24]. Indeed, subsequent research would develop to use refactoring either in the formal sense (e.g., [42, 107, 22, 116]) or in a more pragmatic way that included, for example, unsafe refactorings, bug-fixes and API-level changes (e.g., [67, 108, 72, 35]). The latter, pragmatic kind of refactoring research is often more developer oriented and is the category to which the research in this dissertation belongs.

Despite the influence of Mens and Tourwé's model, it did not accurately describe the workflow that developers use. Steps 1 and 2 were commonly thought of as the detection and removal of *code smells*, that is, of patterns in source code that makes the code difficult to work with, for example due to overly long methods or too deep nesting of logical structures [46]. However, later research into how developers refactor (Chapter 2.4-2.5) found that this is not the main motivation for the application of refactorings. Instead, developers rather apply refactorings frequently, in small steps, and often interleaved with other changes [87, 126, 67, 91] for the purpose of preparing or completing functional software changes [112, 74, 95]. Muprhy-Hill et al. [86] coined the term "floss-refactoring" for these types of changes and found that developers often apply them during functional changes to avoid introducing smells in the first place. Roberts, Brant, and Johnson also describe this workflow in their original paper on the Refactoring Browser [104]. However, some of the early works on refactoring tool usability was made under the assumption that the tool should automate the process that Mens and Tourwé had laid out (e.g., [77]), which includes locating and removing code-smells. In contrast, developers were refactoring to prevent the smells from ever occurring.

These discrepant workflows can give rise to a number of the usability problems once developers try to replace their manual process with existing refactoring tools. For a simple example, consider a refactoring tool that is designed to ensure that program behavior is preserved. When a developer invokes this tool, it might check behavior-related conditions and even present the developer with information about any potential changes to the program's behavior. However, the developer might be applying the refactoring with the intention to progress towards a software change goal, and consequently, they may be concerned both with the behavior changes but also with how the refactoring tool will impact subsequent steps in their workflow. They may, for example, worry that the tool will prematurely remove information that they need to understand other areas of the code, or that the tool will make it more difficult to recover impacted areas later in their workflow. Such usability problems can occur even with tools that are correct.

2.3 Refactoring Tool Usability

The first investigation of refactoring tool usability was done by Mealy and Strooper in 2006, five years after the first refactoring tool crossed Fowler's Refactoring Rubicon by supporting `extract-method`. The reason why usability of refactoring tools is important is because the enticing promise of refactoring to make software evolution faster and cheaper by order an order of magnitude [120] is realized only if developers use the tools that are available to them. If the tools have usability problems, developers may fail to use these tools or fail to achieve such gains when trying to use them.

Mealy and Strooper performed a theoretical evaluation of the usability of six refactoring tools for Java by comparing their capabilities to the steps in Mens and Tourwé's model of the refactoring process [76]. They concluded that "usability of refactoring tools requires further research/consideration". Next, Mealy et al. operationalized the ISO 9241–11 interface design standard [4], again using Mens and Tourwé's refactoring model, and distilled a list of 81 usability requirements for refactoring tools [77]. While this work was an important early investigation into the usability of refactoring tools, the premise of the refactoring process was later found to be subtly different. For example, it was found that developers largely do not use refactoring tools to remove smells, but rather as part of functional code changes. Thus, some of the requirements formulated by Mealy et al. are less relevant to tool usability than first assumed.

For example, one requirement from Mealy et al. is that the tool must "[a]utomate code-smell identification" [77], which corresponds to Step 1 in Mens and Torwé's model [78], so for a tool to fully support their model, this is indeed necessary. However, later re-

search has found that refactoring is largely performed as part of functional changes and for preventative purposes rather than to remove existing code-smells [112, 87, 95]. Consequently, this usability requirement is not necessary for a tool that supports refactoring.

Another guideline is that the tool must "[a]llow suggested changes (to the system being refactored) to be viewed, changed, or cancelled prior to transformation" [77]. Interestingly, this guideline is in line with current refactoring tools and the practice of offering the user a Preview View and a Configuration View before the transformation is applied (Chapter 2.1). However, no studies have shown the effectiveness of Preview Views. In fact, early studies of developers found that many developers do not inspect these views [126] a recent survey of developers finds that many developers do not find these views effective even today [51].

I take a similar approach to Mealy et al. in Chapter 4 where I operationalize the usability framework put forth in ISO 9241-11 [5]. However, the approach I took is different in the following aspects: 1) I based the operationalization on recent findings into how and why developers refactor that differ significantly from the process put forth by Mens and Tourwé [78] and that Mealy et al. relied on, and 2) I refine the resulting framework based on experiences from practitioners and distill a theory of refactoring tool usability that can be reused by researchers and practitioners, rather than creating specific usability requirements and performing a theoretical evaluation of tools.

2.4 Studying Developers' Refactoring Experiences

While the early research into refactoring tools was largely based on a theoretical understanding of how refactoring could be used to evolve software [121, 35, 94], later research began investigating how developers use refactorings. Learning how developers apply refactorings manually or how they use existing tools can enable the development of more effective and satisfying tool support. For example, by learning how often individual refactorings are applied, toolmakers can prioritize automation of the most frequently performed refactorings, and by learning ways in which existing refactoring tools are inadequate, toolmakers can improve these aspects of the tools. For these reasons, it is interesting to learn about how developers refactor and whether they use refactoring tools. If they use tools, it is useful to know how they experience these tools, and if not, it is interesting to learn why they are avoided.

There are multiple ways to investigate these questions. One way is to *ask* developers about their refactoring habits through surveys or interviews. Another is to *observe*

developers as they change software; for example through on-site observational studies, laboratory studies, or by recording invocations of refactoring tools in their IDEs. One can also *mine* refactorings from source code changes, such as the ones found in open source code repositories.

2.4.1 Asking Developers

The approach of surveying and interviewing developers is a popular way of learning about their refactoring behavior. For example, Kim et al. surveyed developers at Microsoft [67], Sharma et al. surveyed software architects at Siemens Corporate Development Center India [108], Murphy-Hill and Black surveyed participants at the Agile Open Northwest conference [89], Leppänen et al. interviewed software architects and developers from Finnish software companies [72], and Tempero et al. surveyed a large number of software developers [118]. Both approaches to collecting information have merits and drawbacks.

Surveys provide a scalable and cost-effective way of collecting information from developers. It is often preferred due to the high number of responses that can be collected. However, these surveys are usually kept short and brief and consequently, questions might be kept somewhat shallow. For example, surveys provide limited possibility for follow-up questions. In contrast, interviews tend to be conducted in person and are more time-consuming and less convenient for both the researchers and the subjects. However, they afford researchers the opportunity to explore the subject more in-depth and to ask clarifying questions from the subjects. A mix of the two can also be utilized by performing "interviews" via email to inquire about, for example, source code changes that the subject has performed during a study [87] or in open-source repositories [112].

In the context of research into refactoring, there is an important limitation of both surveys and interviews. Any methodology that requires developers to respond to questions about their refactoring and experiences with refactoring tools, suffers from a potential terminology gap in the use of the term "refactoring".

The researcher might use the term "refactoring" to refer to a change pattern from a catalogue [46], or to only special kinds of code-editing activities that preserve the program's behavior [94, 98]. Meanwhile, the participant might consider "refactoring" to refer to bug-fixes, library-upgrades, and to larger architectural changes such as turning a monolithic software application into micro-services [67, 72, 108, 118, 98]. Leppänen et al. describe their participants' usage of the term in this excerpt from their paper: *"Interviewees usually reserved the word "refactoring" for restructuring and redesigning the system, as in preparatory or planned refactoring. They didn't consider daily small*

changes—for example, method renaming and moving the code around a bit—as refactoring." [72] A similar observation was made by Kim et al. [67] in their study of Microsoft developers. One of their participants stated: *"[Refactoring is] changing code to make it easier to maintain. Strictly speaking, refactoring means that behavior does not change, but realistically speaking, it usually is done while adding features or fixing bugs."* As a result, developers may agree with statements like this one *"Refactorings supported by a tool differ from the kind of refactorings I perform manually."* [67] because they are referring to the lack of tools for micro-service migration.

Moreover, practitioners may be unfamiliar with most or all refactorings that are supported by tools [51]. Consequently, findings from such studies are limited because practitioners may respond based on their interpretation of the concept of refactoring or their awareness of tool support. For example, a developer may estimate that they perform 80 % of their refactoring with tools while the real number is 10 % based on the existing tool support.

2.4.2 Observing Developers

Observational studies offer an alternative to surveys and interviews, and have the benefit that the *actual* developer behavior is collected rather than their self-reported behavior. In the past, these studies were time-consuming because, similarly to interviews, an observer needed to be present. Today, advances in the techniques for collecting interaction data from IDEs enable large-scale observational studies of refactoring "in the wild". For example, the Mylyn monitor[4] has been used in multiple such studies of refactoring tool use [86, 126]. This enables investigating how developers interact with refactoring tools that are available to them in their IDEs during their usual work tasks. Through such studies, researchers can recognize cases of repetitive invocations, cancellations, and reversions of refactorings as well as common configurations and patterns of batch applications [125, 91, 87].

However, when observations are made during regular work tasks, it can be difficult to compare between developers because they might use different IDEs, work on different codebases, and use different tools and plugins. Moreover, there might be difficult to get access to developer's environments due to concerns about security and code ownership. If an observational study is conducted as a laboratory study, then it is challenging to create realistic tasks. In those cases, the tasks used in previous work has typically investigating refactoring tool use in isolation from other tools, on subgoals that are taken out of context of an overall change goal (e.g., [84, 50]). While such studies provide valuable

[4] `https://www.eclipse.org/mylyn/`, accessed August 14, 2021

insights and enable improvements to individual tools, the improvements are not always compatible with the use of the tool within a larger change. It is also non-trivial to merge these disparate study findings into a framework that is shared between tools, since they focus on different tasks and experience measures.

2.4.3 Mining Source Code Changes

Mining studies have become popular ways to quantify the occurrences of individual refactorings and evaluate their impact on quality measures. With the increasing popularity of open-source version control systems, it became possible to apply mining tools to the source code histories that are stored in these repositories. RefactoringMiner [123] is a state-of-the-art tool for this purpose and the tool that I used to locate the commits that I used to build tasks for my laboratory study. By developing algorithms that detect refactorings in these version histories, researchers could investigate how refactorings occurred "in the wild" on a larger scale than before.

By combining this with the aforementioned approaches, researchers can even investigate the degree of use and *disuse* of refactoring tools: if a refactoring is detected both in the version control history and in the IDE, then the tool was used, whilst if a refactoring is detected in the version control history and *not* in the IDE, then the tool was not used. Occurrences of the latter can be interpreted as a missed opportunity for the developer to use the tool.

Unfortunately, even when these mining-based studies focus on tool-supported refactoring operations they rarely involve the developers who performed the study or the experiences they had. Only a few studies have combined repository mining with information from developers' IDEs make for interesting insights about potential and actual tool use [86, 87, 91, 126]. However, these studies largely do not involve developers. In contrast, Silva et al. contacted developers who made the commits to ask them about the changes they made [112].

2.5 Studies of how Developers Refactor

Researchers have employed multiple methods to learn about how developers apply refactorings and how they use and experience refactoring tools. These studies show that a number of refactoring tools are in *disuse*, such as `move-method`, `inline-method`, and `remove-parameter`, and that developers are often perform refactorings in the context of

funcational changes.

Murphy-Hill, Parnin, and Black [87, 86] aggregated and analyzed data from several datasets: the Eclipse Usage Collector dataset of anonymized Eclipse IDE interactions, the version control history of the open-source projects Eclipse and JUnit, the refactoring histories from four developers who maintain Eclipse's refactoring tools, and volunteer IDE event data from 41 developers that was collected with the Mylyn monitor and compared with the version control history of the source code these developers were working on. They used this information to characterize how often different refactoring operations were invoked, completed, and integrated into the code repository. Through their analysis, they found that developers' refactoring behavior differs from the process that Mens and Tourwé described (so-called *root-refactoring*). Instead, most of the refactorings that they detected were small and interleaved with other code changes (*floss-refactoring*). Moreover, 90 % of the refactorings that they detected were performed manually, such as `move-member`[5], `inline-method`, and `remove-parameter`. That makes these refactorings interesting to study in order to learn *why* developers avoid using the tools to apply them, and what they do instead. Other researchers have also observed disuse of the same refactoring operations [126, 67, 112].

Vakilian et al. investigated refactoring tool use by analyzing data from IDEs that captured developer interactions. They investigated how developers invoked, configured, and applied refactoring tools and found further evidence for disuse [126]. Based on their findings, the authors conclude that refactoring tools should automate small, predictable code changes that developers can manually compose into complex changes [127].

Negara et al. also combined the analysis of version control histories and tool interactions to study how 23 developers applied refactoring during their daily tasks [91]. They found that over half of all refactorings that they detected to be performed manually and that 30% of them never reached the version control system and corroborated many of the findings from the studies by Vakilian et al. [126] and Murphy-Hill et al. [87], such as that developers prefer to automate small changes and that many refactorings that are applied in an IDE never reach the version control system. Their findings further indicate the need for refactoring tool support with the appropriate usability support and trade-offs for use in software changes.

Silva et al. used an interesting methodological approach to investigate why developers use refactorings [112]. They detected refactoring commits to a number of GitHub open-source projects and immediately query contributors for the motivations behind their

[5]This refactoring includes both moving member methods and fields but excludes moving static methods and fields.

refactoring operations. They found that refactoring activity is mainly driven by changes in requirements or *functional* changes. They also ask contributors if the refactoring was performed manually or using a tool. Over half of the respondents report doing it manually, including for `move-method`, `inline-method`. This shows that refactoring tool usability must be studied in the context of functional changes.

Kim et al. were also interested in how software developers experience refactorings [67]. They conducted a survey of Microsoft employees and more in-depth interviews with a specific refactoring team and an analysis of that team's version history. Their participants self-reported doing 85% of refactorings manually. In their study, the meaning of the term 'refactoring' was broader and refactoring operations like `remove-parameter` was used interchangeably with large quality-improving program changes that occurred over several years. The latter category of refactorings is less amenable to direct tool support and varies substantially from the smaller operations for which automated support appears in development environments. This definition of refactoring is nonetheless common in industry and can lead to ambiguity in study results when the type of refactoring is not defined. In this study, participants also self-reported performing `remove-parameter` and `inline-method` mostly manually. The survey does not consider use of `move-method`.

Liu et al. were interested in what motivates developers to apply refactorings and focused their study on the `extract-method` refactoring. Through an analysis of version histories of open-source projects and interviews of developers, they found reuse to be the main motivation for performing this specific refactoring [74], but did not consider how the refactoring was applied or how the developers experienced it. Paixão et al. also investigated why developers apply refactorings and mined code changes from open-source communities, finding that refactorings are mainly driven by the intent to introduce or enhance features [95]. In both cases, developers perform refactorings with a larger goal in mind, rather than in order to remove smells.

Three other studies that summarize perspectives on refactoring from the industry are conducted by Sharma et al. [108], and Leppänen et al. [72], and Tempero, Gorschek, and Angelis [118]. Through interviews and surveys with industry practitioners, the authors investigate hinderances to refactoring adoption and how practitioners experience refactoring [72, 118, 108]. All the studies find that refactoring tools are not adequately supported or adopted by practitioners. Sharma et al. concludes that *"academia, industry, and tool vendors must collaborate to create a new generation of more useable, effective refactoring tools."* [108]

All of these studies found refactoring to be a frequent practice in the evolution of software. By mining repositories, extracting data about tool use from IDEs, and surveying and

interviewing practitioners, these studies show that refactoring frequently occurs, and that refactoring tools are often not used. Instead, developers perform most refactorings manually. These findings indicate that there is a high potential for automation which can save developers from the time-consuming and error-prone manual code changes.

2.6 Factors that Impact the Use and Disuse of Refactoring Tools

Numerous studies have found that refactorings frequently occur as part of software changes. Moreover, they also found that the refactoring tools that, in theory, can automate these code changes are underused by developers. It is interesting to understand *why* this underuse occurs and which factors contributes to it.

Murphy-Hill et al. investigated why developers that they had studied avoided using refactoring tools by performing followup surveys of 5 of these participants [86, 87]. Vakilian et al. took a similar approach and performed followup interviews with 9 participants to learn why they avoided using refactoring tools [126]. Silva et al. also found several factors that impacted whether the authors of repository commits that they surveyed had used refactoring tools or not [112]. Kim et al. speculate about factors based on qualitative analysis of their participants' comments [67]. Negara et al. did a quantitive investigation of automated refactoring versus manual refactorings and speculated about factors that could cause their findings [91]. In addition, Leppänen et al. [72], Sharma et al [108], and Tempero et al. [118] report on some factors that prohibit refactoring tool use based on their studies.

Table 2.1 summarizes the factors that were found to impact refactoring tool use or the disuse of these tools in these studies. As shown in the table, many of the same factors occur in multiple studies. Therefore, I organize this section around factors rather than studies.

Awareness is a factor that occur across multiple studies. Murphy-Hill et al. refer both to the developer's awareness of the refactoring tool's existence and to their awareness of performing refactorings [87]. For example, a developer may avoid using a tool to perform `inline-method` because they are unaware such a tool exists. Negara et al. also speculate that novices are less *familiar* with refactoring tools than experienced developers and therefore use them less [91]. On the other hand, a developer may be aware of this tool's existence but completes the corresponding code changes manually before they become aware that they are performing an `inline-method` refactoring that

Table 2.1: This table contains an overview of the studies that have investigated developers' adoption of refactoring tools. For each study, the middle column lists the percentage of refactorings which are found to be manual in that study and the rightmost column lists the factors that the authors report prohibit the use of refactoring tools. If a study did not report this information, the column contains a dash.

Source	Manual	Factors that prohibit tool use
Murphy-Hill et al. [86, 87]	90 %	Awareness, Trust, Opportunity, Touch Points, Disrupted Flow
Murphy-Hill et al.[84]	-	Inability to select code, Incomprehensible error messages
Vakilian et al. [126]	> 50%	Awareness, Trust, Predictability, Naming, Need, Configuration
Silva et al. [112]	> 50%	Awareness, Trust, Complexity, IDE Support, Familiarity
Kim et al. [67]	85%	IDE support is too low-level, Lack of support for the types of refactoring developers want to make
Negara et al. [91]	> 50%	Familiarity, Speed
Leppänen et al. [72]	-	Trust
Sharma et al. [108]	-	Lack of exposure, Finding tools, Differential code base
Tempero et al. [118]	-	Lack of information identifying consequences, Lack of certainty regarding risk, Difficulty translating a refactoring goal into refactoring operations

could be automated. Vakilian et al. and Silva et al. also suggest that lack of awareness or familiarity with reafctoring tools prohibit tool use [126, 112]. In addition, Silva et al. also describe participants who lacked *familiarity* with the refactoring capabilities of their IDEs [112]. In the same vein, Sharma et al. refer to *lack of exposure* and *finding tools* as factors that prohibit refactoring tool use [108]. Tempero et al. refer to "difficulty translating a refactoring goal into refactoring operations" which might be hindered by the developer's awareness or familiarity with the tools that are available to them [118]. Lack of awareness and lack of familiarity can both be mitigated by education and by developing tools that automatically recognize that a developer is performing a manual refactoring. Examples of such tools are BeneFactor [50] and WitchDoctor [45].

Trust is described as a function of usability and reliability by Vakilian et al. [126] who report that *"[w]e found usability to be a more important factor than reliability on users' trust in a mature refactoring tool like that of Eclipse."* [126, 238] Interestingly, it is unclear how to address trust nor what aspects of a tool impacts developers' trust. Murphy-Hill et al. describe trust as whether developers have faith in the refactoring tool; lack of trust indicates a fear of introducing errors or unintended side effects [87]. They propose to present the user with inspection checklists [86] to improve trust. This is similar to the Preview View that Campbell and Miller [28] propose to address the developers' uncertainty of how a tool will change their program or a fear that it will "mangle their code" [28, p. 1]. Campbell and Miller refer to "predictability" rather than trust, despite describing similar fears to the ones Murphy-Hill et al. propose as related to trust. Leppänen et al. mention that their participants "have little trust in automation" but do not elaborate on what that mean or why [72, p. 68]. Brant and Steimann [24] discussed whether tool correctness is necessary for developers to trust and use a tool and do not reach a conclusion. Tempero et al. list "lack of certainty regarding risk" as a factor that prohibits tool use [118].

Predictability indicates the developer's ability to *guess* the result of applying the tool [126], i.e. how it will change their code (similar to "trust" in [28]). Tempero et al. also refer to "[l]ack of (..,) information identifying consequences" [118, p. 60] as a factor that prohibits tool use. Oliveira et al. investigated the predictability of refactoring tools by conducting a survey where developers choose which output they expect from refactoring operations (e.g., `Rename Field` and `Pull Up Field`) and found that respondents rarely agree about what to expect nor do their expectations align with the behavior of their IDEs [92]. Developers' inability to predict a refactoring's outcome might also impact their trust in the tool. Vakilian et al. [126] point out that predictability is impacted by complexity: a more complex refactoring is harder to guess the outcome of and, if it impacts many locations, is harder to verify. Simple, local refactorings, in contrast, are easier to predict and verify. They suggest. the ability to preview the refactoring as

a way to improve predictability. However, Murphy-Hill et al. [86] speculate that developers find preview windows insufficient to understand the refactoring's *touch points*, i.e., the set of places in the code that the refactoring would effect. They also describe that preview windows *disrupt* developers' *flow* when requiring context-shifts into a separate window.

Configuration options afford users of refactoring tools control over the transformations that the tool will perform on their code. For example, the `change-signature` in IntelliJ and Eclipse can be invoked on a method and then be configured to remove or add parameters, change the method's return type, or even the parameter names (Chapter 2.1). Vakilian et al. report that configuration windows introduce additional cognitive overhead and disrupt developers' flow [126]. Murphy-Hill report that developers rarely configure refactorings and instead prefer to tweak the resulting code [87].

Need indicates that the developers are performing code changes that a tool can automate [126]. Some tools are not used because the operations they automate are rarely appropriate. This echoes *opportunity* in [86]: the situation in a developer know that the refactoring tool exist, but they do not recognize opportunities to apply it. Similarly, in Silva et al.'s work, if a refactoring has too low of a complexity, participants indicate that there is no need to use a tool [112]. These contradicting observations indicate that some tradeoff must be made in the design of tools.

Naming indicates that hard-to-understand names of refactoring tools act as a barrier for developers in finding and trying tools [126]. This was also briefly mentioned in [86] and addressed in [88] with gesture-based menus. Other works that has approached this problem is Lee, Chen and Johnson with Drag-and-Drop invocation of refactorings [71] and several works that can recognize a manual refactoring and invoke refactorings to complete it [45, 50, 100]. Some of these efforts aimed at addressing this factor overlap with the efforts related to *awareness* and *familiarity*, such as the detection and completion of manual refactorings. Toolmakers can also address this factor by simplifying refactoring *names*. One example of this is the decision to keep only high-level names like `move` and `change-signature` in the refactoring menu of IntelliJ and Eclipse, and then use techniques like Configuration View and automated detection of program elements to determine the exact refactoring (Chapter 2.1). However, this can cause other problems, such as the effort required to configure refactorings and an inability to understand how to invoke the right refactoring.

Complexity is both used to explain avoidance of tools due to lack of *need* to automate overly simple refactorings [112] and the difficulty of using overly complex refactorings [126]. Multiple studies have found that developers more often automate simple

refactorings and perform complex refactorings manually, even for refactorings that are supported by refactoring tools [91, 126, 87].

IDE support indicates that the IDE does not support a refactoring operation [126]. Lack of *awareness* or *naming* may contribute to developers falsely claiming this as a reason. However, many of the refactoring activities that developers perform may not be supported by IDEs. This can be mitigated by implementing support for more refactoring operations and support for a wider variety of programs. However, this might negatively impact the *complexity* of tools, the *predictability* and *trust* of tools, and developers may still not have *awareness* of these additional operations.

Differential codebase is listed as a factor by Sharma et al. [108]. They explain this as follows: *"We want to focus on assessing and improving only the changed or newly added parts in the source code, and refactoring tools can't run only on this differential code base"* [108, p. 47]. This is an interesting problem, and one that has not beed addressed in research that proposes tool improvements.

Some factors relate to specific usability problems that arise from the tools' user interface. Murphy-Hill and Black [84] investigated usability problems with the `extract-method` refactoring tool and found that developers had problems selecting the right code segments and problems understanding error messages. They acted on this information by proposing tools that could aid in making a correct code selection and proposing more understandable error messages. Several of the specific changes that they proposed, such as more understandable error messages for `extract-method`, were incorporated into the Eclipse refactoring tool.

While previous research has identified multiple factors that prohibit the use of refactoring tools, acting on these factors to improve the adoption of tools is not a trivial matter. Some factors can be isolated and addressed. While some factors can be isolated and addressed individually, others are tangled, difficult to define and address, and in some cases contradicting. For example, *awareness* can be tackled through education and support of invocations through manual changes and *IDE support* can be improved by increasing the number of refactorings that an IDE supports. Meanwhile, *trust* and *predictability* have been persistent topics in the research of refactoring tool usability, yet there is no agreed-upon definition of exactly what makes refactoring tools trustworthy or predictable.

Addressing such tangled and ill-defined factors are complicated by their contradictions. For example, a toolmaker might be able to improve *IDE support* and make a tool more flexible by increasing the number of situations a tool can work in and even accept some erroneous invocations. However, in doing so, the tool's *trustworthiness* might be reduced

because it will occasionally introduce errors. As another example, a toolmaker might increase the tool's *trustworthiness* and *predictability* by limiting the accepted invocations to programs that can be easily analyzed and transformed. However, that might reduce the *IDE support* because there may be many situations in which the tool can not be applied. Refactoring tools cannot always both meet developers desires and be fully correct [24]. As a final example, *naming* and *awareness* can be addressed by simplifying the invocation of refactorings through automated detection and simplified names. That, however, might result in less *predictable* tools as the developer might accidentally invoke the wrong refactorings. Such design tradeoffs must be made by toolmakers who wish to improve the usability of refactoring tool. This illustrate the challenges in addressing factors that prohibit the use of refactoring tools.

2.7 Software Changes

As refactoring tools support developers' work on software change tasks, one might expect studies of software changes to provide insights into how refactoring tools are used and experienced during these activities. Unfortunately, studies of software change tasks tend to focus on such aspects as how software artifacts are altered through the change process or the questions developers ask as the change progresses, without investigating the role of tools as part of the change process (e.g., [111, 96]).

The need for software tools to help support change to software systems has been recognized for many years (e.g., [65]). While the need for tools has been evident, our understanding of what kinds of tools are needed, how those tools function and how developers can best match tools to tasks is still evolving.

Early software change studies focused primarily on how developers approach comprehending the software they must change. Researchers developed models to try to explain how individual developers comprehend source code (e.g., [114, 97, 129]). At the time these models were developed, tools such as textual search and compilers were already prevalent. However, these models focused on human cognition models without considering how tools interplayed with the comprehension process. Flor and Hutchins took a different approach, considering programming as a complex activity in which multiple developers work collaboratively on maintenance tasks [44]. They described a distributed cognition model, but they also described the role tools played for developers in their study in helping them enact one out of several alternative strategies to tasks in their study. Although these insights were not drawn out as major findings, their work marks a consideration of how tools and tasks interact. Our work continues this direction of

inquiry.

Drawing on the importance of program comprehension for software change tasks, researchers have honed in on specific activities that form part of a change task and the specific tools used to support those activities. For example, Starke et al. studied how programmers use search tools as part of program comprehension activities undertaken during corrective maintenance tasks [115]. Fleming et al. used *information foraging theory* to evaluate tools for debugging, refactoring and reuse tasks [43] and found that these tools can alleviate the foraging activities that developers otherwise do manually.

As research on software change progressed, there became an increasing focus on understanding developer work practices during software change tasks, considering the challenges developers faced and how to support those challenges with tools. Singer et al. report on the work practices of professional developers and use insights from the discovered practices to design a tool to support software maintenance [113]. Ko et al. used a laboratory-based experiment to study the different approaches of multiple developers to corrective and perfective maintenance tasks and used insights gained to suggest design requirements for tooling aimed at supporting software maintenance tasks [68]. Sillito et al. conducted a laboratory-based study to consider the goals developers formed when performing software change tasks and how developers used tools available to perform those goals. From these observations, they described opportunities for further tool development to better support the developers [110]. However, none of these works has focused on refactorings as part of the change tasks.

Rajlich presented a phased model of the software change process that included refactorings as part of functional changes [99]. Their work, however, related to a higher-level view of these activities and did not include descriptions of how developers interleave the use of tools in these phases or how refactoring tools can better support them. In an unpublished paper, Wilson et al. [130] considers how the change propagation tool JRipples [27] supports such activities. JRipples uses code dependencies to support *change impact analysis* [21] and *change propagation* [26] by providing the developer with a checklist of locations in the code that might affected by a change. Buckley et al. has identified refactoring tools as another means of change propagation [26], but rather than using any code dependencies, refactoring tools propagate changes according to specific rules [46]. Despite the similarities between these types of tools, they are not typically compared in studies.